

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_14

14. Bridge Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Bridge pattern. You'll see two different implementations.

GoF Definition

It decouples an abstraction from its implementation so that the two can vary independently.

Concept

This pattern is also known as the Handle/Body pattern. Using it, you decouple an implementation class from an abstract class by providing a bridge between them.

This bridge interface makes the functionality of concrete classes independent from the interface implementer classes. You can alter different kinds of classes structurally without affecting each other. This pattern initially may seem to be complicated, which is why, in this chapter, I've provided you with two different implementations containing lots of explanations. The concept will be clearer to you when you go through the examples.

Real-Life Example

In a software product development company, the development team and the marketing team both play crucial roles. Normally, the marketing team does a market survey and gathers customer requirements. The development team implements those requirements in the product to fulfill customer needs. Any change (say, in the operational strategy) in one team should not have a direct impact on the other team. In this case, you can

think of the marketing team as playing the role of the bridge between the clients of the product and the development team of the software organization.

Computer World Example

GUI frameworks can use the Bridge pattern to separate abstractions from the platform-specific implementation. For example, using this pattern, you can separate a window abstraction from a window implementation for Linux or macOS.

In Java, you may notice the use of JDBC, which provides a bridge between your application and a particular database. For example, the `java.sql.DriverManager` class and the `java.sql.Driver` interface can form a bridge pattern where the first one plays the role of abstraction and the second one plays the role of implementors. There are many concrete implementors such as `com.mysql.cj.jdbc.Driver` and `oracle.jdbc.driver.OracleDriver`.

Note The online link <https://dev.mysql.com/doc/connector-j/8.0/en/connector-j-api-changes.html> confirms that the name of the class that implements `java.sql.Driver` in MySQL Connector/J has changed from `com.mysql.jdbc.Driver` to `com.mysql.cj.jdbc.Driver`. The old class name has been deprecated.

In my other book, *Interactive Object-Oriented Programming in Java* (Second Edition, Apress, 2020), I have a chapter on database programming. Here are some lines from it for your immediate reference.

DriverManager: This *class* manages a set of JDBC drivers. It matches the connection request from a Java application with the proper database driver. (It is important to note that JDBC 2.0 provides an alternate way to connect to a data source. The use of a `DataSource` object is a recommended way to connect a data source.)

Driver: This is an *interface* to handle the communication with the database server. Each driver must provide a class that will implement this interface. Each `Driver` class should be small and standalone so that it can be loaded without vast supporting code. When a `Driver` class is loaded, you should create an instance of itself and register with `DriverManager`.

In that chapter, to connect to a MySQL database, I used the following lines:

```
Class.forName("com.mysql.cj.jdbc.Driver").newInstance();  
Connection connectionOb = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/test",  
    "root", "admin");
```

Similarly, to connect with an Oracle database, you can use the following line of code:

```
Class.forName("oracle.jdbc.driver.OracleDriver").newInstance();
```

To connect with a MS SQL Server, use the following line of code:

```
Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver").newInstanc
```

Connection: This *interface* provides the methods to connect a database. Your SQL statements execute, and the results are returned within the context of a connection. All communication with the database passes through the connection object.

getConnection() : This method attempts to make a connection to the given database URL. Multiple overloaded versions are available for this method.

Implementation

Suppose you need to design a piece of software for a seller who sells different electronic items. For simplicity, let's assume the seller is currently selling televisions and DVD players, and they sell them both online and offline (in different showrooms). To implement this, you start with the designs shown in Figure [14-1](#) or Figure [14-2](#).

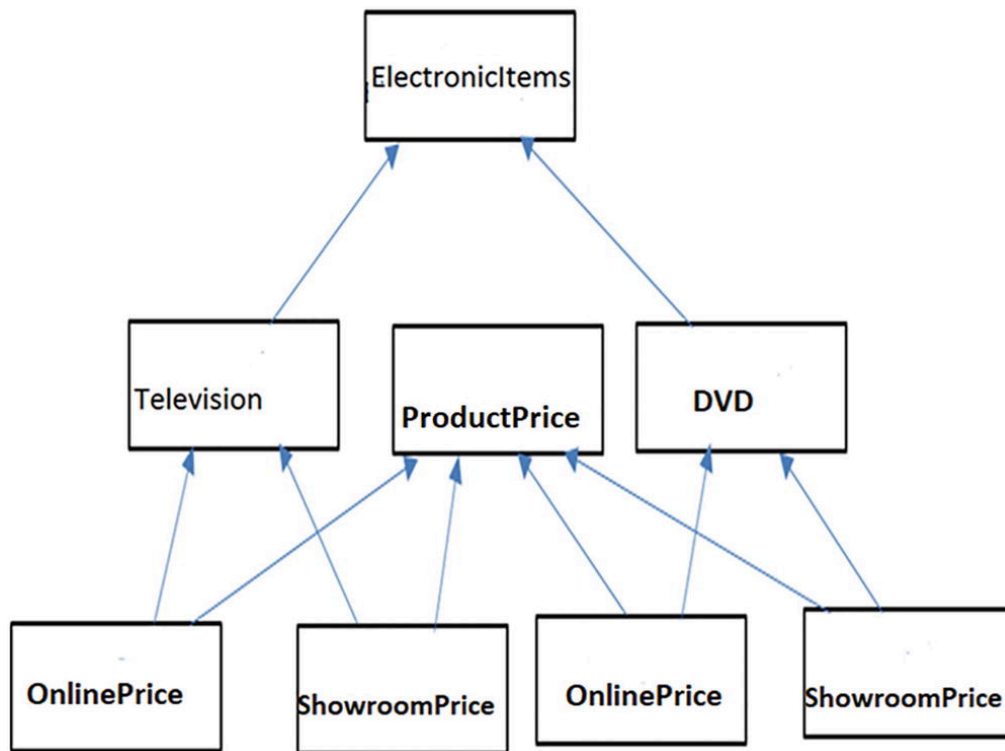


Figure 14-1 Approach 1

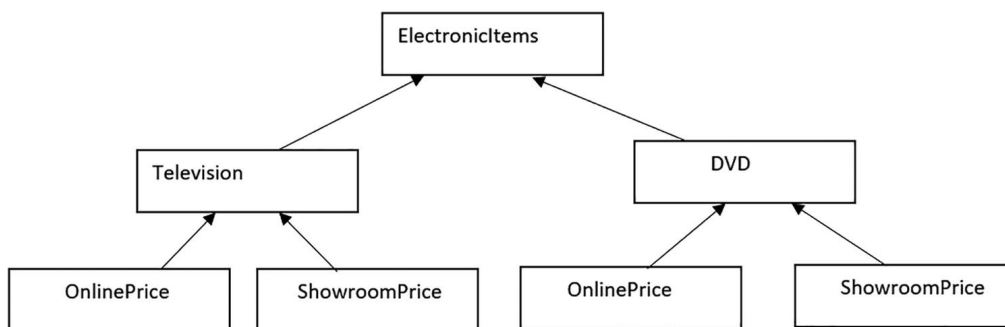


Figure 14-2 Approach 2

On further analysis, you discover that approach 1 is messy and difficult to maintain.

In the beginning, approach 2 looks cleaner, but if you want to include new prices such as `ThirdPartyPrice`, `FestiveTimePrice`, and so on, or if you want to include new electronic items like air conditioners, refrigerators, and so on, you will face new challenges because the elements are tightly coupled in this design. But in a real-world scenario, this kind of enhancement is often required.

So, you need to start with a loosely coupled system for future enhancements, so that either of these two hierarchies (electronics items and their prices) can grow independently. The Bridge pattern is perfect for such a scenario. When you use the Bridge pattern, the structure may look like the one in Figure [14-3](#).

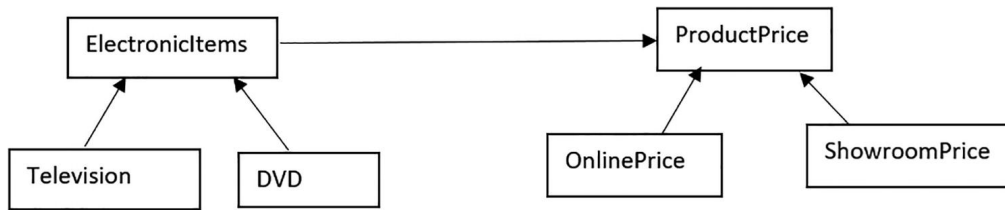


Figure 14-3 Maintaining two separate hierarchies using the Bridge pattern

The idea behind this kind of structure is that you maintain two different inheritance hierarchies. They can grow or shrink without affecting each other. Initially, it may sound strange to you, but once you see a sample implementation, the idea behind this philosophy will be clearer. So, keep reading!

Before I show you a sample implementation, let's start with the most common class diagram of a Bridge pattern, shown in [Figure 14-4](#).

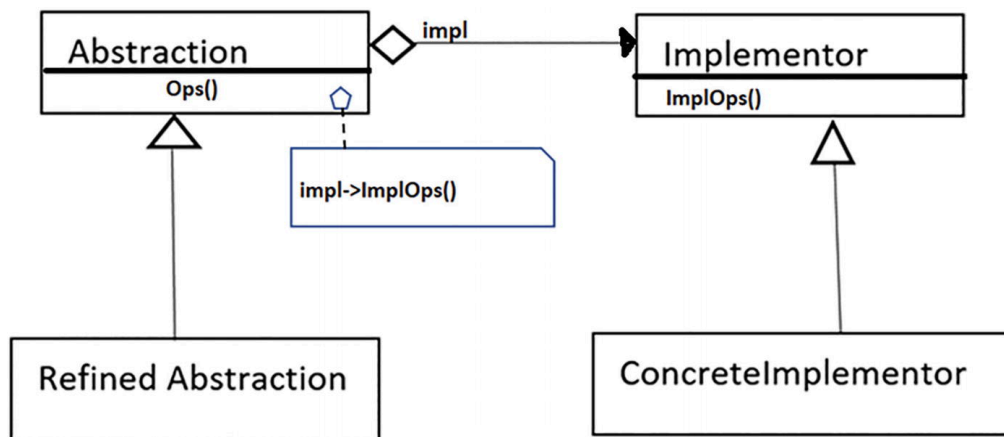


Figure 14-4 A classic Bridge pattern

In this class diagram,

- **Abstraction** defines the abstract interface and maintains the **Implementor** reference. In this example, it is an abstract class, but don't assume that you need an abstract class or interface to define an abstraction. The word "abstraction" here is used regarding the methods that are used to remove the complexity. These methods simply hide the inner details of their working mechanism from the client code. In the example, `ElectronicItem` plays the role of abstraction. The common operations are placed inside this abstraction class.
- **RefinedAbstraction** (a concrete class) extends the interface defined by **Abstraction**. Remember that it is optional if you do not start with an abstract class or an interface. In the example, the `Television` and `DVD` classes are the concrete abstractions.
- **Implementor** defines the interface for implementation classes. The interface methods don't have to correspond to the abstraction meth-

ods exactly. Typically, they include the primitive operations, and `Abstraction` defines the high-level operation based on these primitives. Also, note that there need not be a one-to-one mapping between an abstraction class method and an implementor method. You can use a combination of the implementor method inside an abstraction class method. In demonstration 2, you'll see this clearly, or you can refer to Q&A 12.8. In the example, `ProductPrice` is the implementor interface.

- `ConcreteImplementor` (a concrete class) implements the `Implementor` interface. In the upcoming example, the `OnlinePrice` and `ShowroomPrice` classes are the concrete implementors.

I follow a similar design in the upcoming demonstrations. For your ready reference, I also point out all of the participants in the implementation with supporting comments.

Class Diagram

Figure 14-5 shows the important parts of the class diagram for the upcoming demonstration (demonstration 1). I do not show the full method signatures in order to make the diagram short and simple. But an important note is added and separate hierarchies are marked so that you can notice the heart of the design.

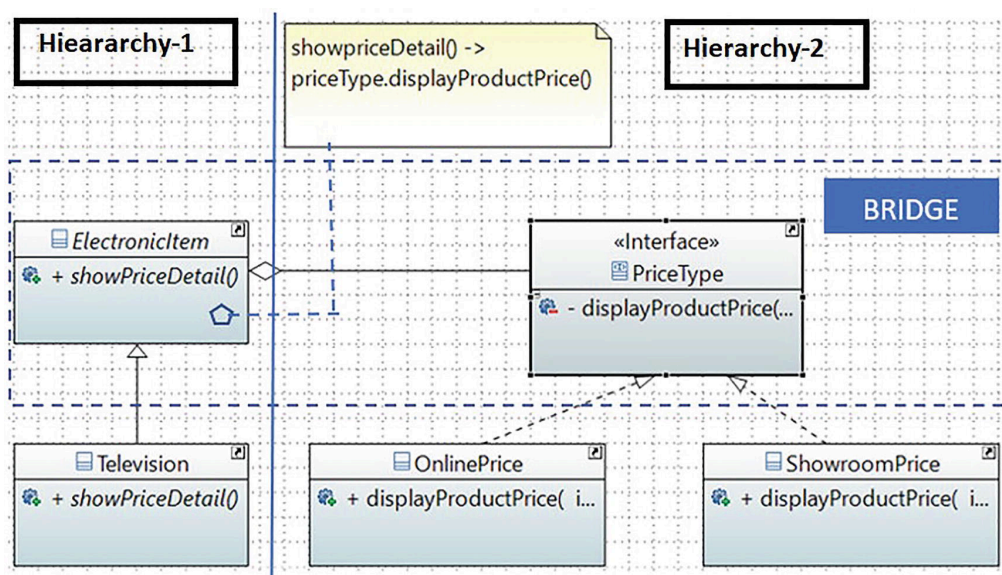


Figure 14-5 Class diagram

Package Explorer View

Figure 14-6 shows the structure of the program.

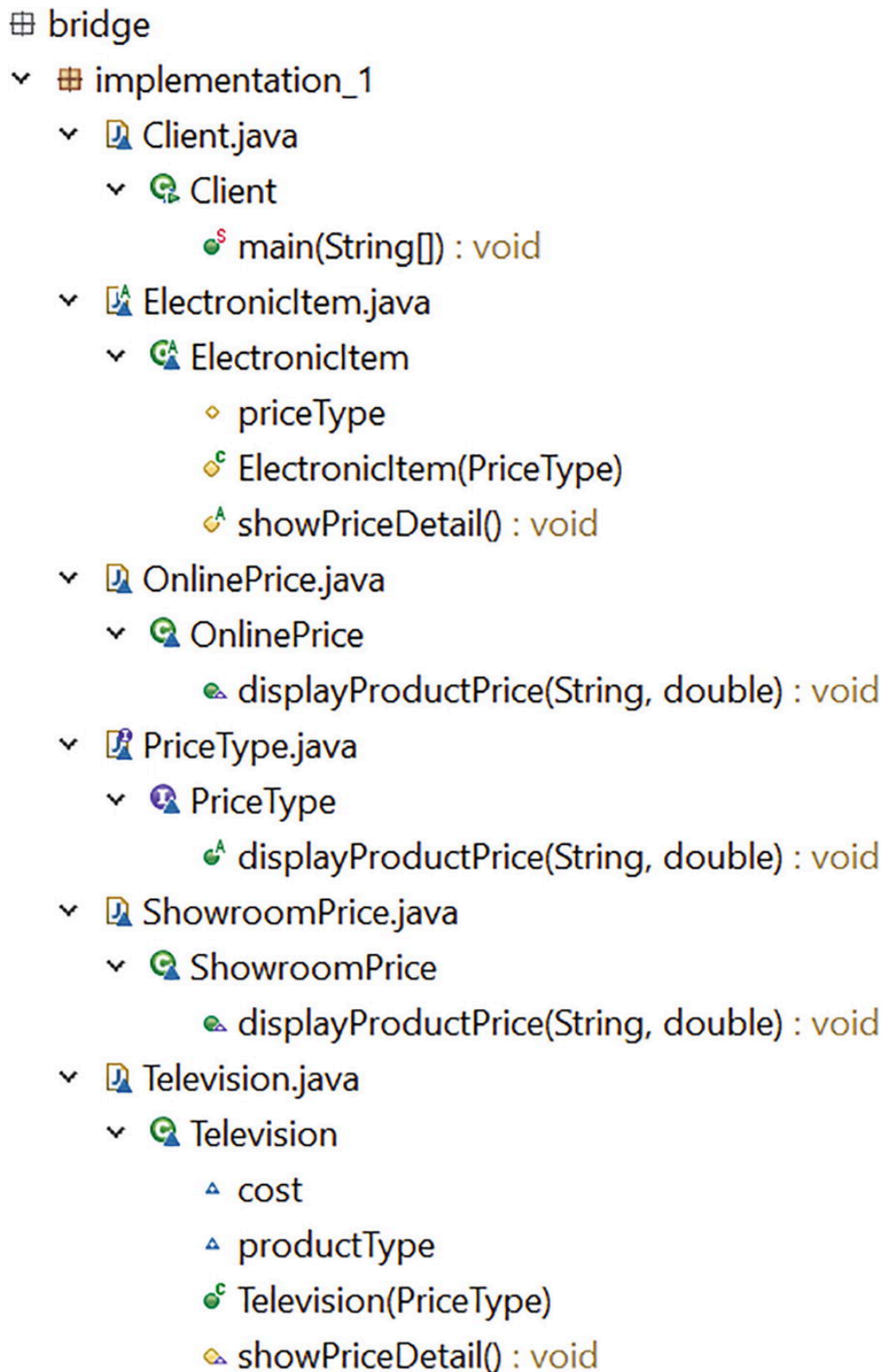


Figure 14-6 Package Explorer view

Demonstration 1

The package `jdp3e.bridge.implementation_1` contains all parts of the program. In this example, `ElectronicItem` is the abstraction class. It is placed in hierarchy-1. This class is defined as follows:

```
// Abstraction
abstract class ElectronicItem {
```

```
// Composition - implementor
protected PriceType priceType;
protected ElectronicItem(PriceType priceType) {
    this.priceType=priceType;
}
/*
 * This method implementation specific.
 * We'll use an implementor object to invoke
 * this method.
 */
protected abstract void showPriceDetail();
}
```

The interface `PriceType` is the implementor interface. It maintains the second hierarchy and is defined as follows:

```
// Implementor
interface PriceType {
    void displayProductPrice(String product,double cost);
}
```

`Television` is the concrete abstraction class that overrides the `showPriceDetail()` method as follows:

```
// Refined Abstraction
class Television extends ElectronicItem{
    String productType;
    double cost;
    public Television(PriceType priceType) {
        super(priceType);
        this.productType="television";
        this.cost=2000;
    }
    /*
     * Implementation specific:
     * We are delegating the implementation
     * to the Implementor object.
     */
    @Override
    protected void showPriceDetail() {
        priceType.displayProductPrice(productType,cost);
    }
}
```

With the supporting comments, you can see that inside the `showPriceDetail()` method, you invoke the `displayProductPrice(...)` method

from the other hierarchy and pass the information about the product type.

The concrete implementors (`OnlinePrice` and `ShowroomPrice`) catch this information and use it in the `displayProductPrice(...)` method. Both concrete implementors are similar. For your immediate reference, here's one of them:

```
// This is ConcreteImplementor-1
class OnlinePrice implements PriceType {
    @Override
    public void displayProductPrice(String productType,
                                    double cost) {
        System.out.println("The " + productType + "'s
                           online price is $" + cost*.9);
    }
}
```

Notice that in the previous code segment, the price is a little bit less (a 10% discount) if a product is purchased online.

Now go through the complete demonstration, which is as follows:

```
// PriceType.java
interface PriceType {
    void displayProductPrice(String product,double cost);
}

// OnlinePrice.java      // This is ConcreteImplementor-1
class OnlinePrice implements PriceType {
    @Override
    public void displayProductPrice(String productType,
                                    double cost) {
        System.out.println("The " + productType + "'s
                           online price is $" + cost*.9);
    }
}

// ShowroomPrice.java    // This is ConcreteImplementor-2
class ShowroomPrice implements PriceType {
    @Override
    public void displayProductPrice(String productType,
                                    double cost) {
        System.out.println("The " + productType + "'s
                           showroom price is $" + cost);
    }
}

// ElectronicItem.java
abstract class ElectronicItem {
```

```

// Composition - implementor

protected PriceType priceType;
protected ElectronicItem(PriceType priceType) {
    this.priceType=priceType;
}
/*
 * This method implementation specific.
 * We'll use an implementor object to invoke
 * this method.
 */
protected abstract void showPriceDetail();
}

// Television.java
class Television extends ElectronicItem{
    /*
     * Implementation specific:
     * Delegating the task
     * to the Implementor object.
     */
    String productType;
    double cost;
    public Television(PriceType priceType) {

        super(priceType);
        this.productType="television";
        this.cost=2000;
    }
    /*
     * Implementation specific:
     * We are delegating the implementation
     * to the Implementor object.
     */
    @Override
    protected void showPriceDetail() {
        priceType.displayProductPrice(productType,cost);
    }
}

// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Bridge Pattern Demo.***");
        System.out.println("Verifying the market price of a television
        // Verifying online price

```

```

        ElectronicItem eItem = new Television(new
                                                    OnlinePrice());

        eItem.showPriceDetail();
        //System.out.println("-----");
        // Verifying the offline/showroom price

        eItem = new Television(new ShowroomPrice());
        eItem.showPriceDetail();
    }
}

```

Output

Here's the output:

```

***Bridge Pattern Demo.***
Verifying the market price of a television.
The television's online price is $1800.0
The television's showroom price is $2000.0

```

Additional Implementation

I include an additional implementation in this chapter to help you explore the flexibility of this pattern. To maintain the previous example, in this example, again I use constructors, not getter-setter methods. It's just a choice. Before I show you the flexibility, let's assume the following:

- The seller can sell either a television or a DVD.
- The seller can provide a discount on a sale in the festive season.
- The seller can provide a double discount for a DVD sale, but he does not wish to provide the same for a television purchase.
- The seller can also send a thank you message once the product is purchased.

As per these new requirements, let's make some changes to the earlier implementation. Let's analyze these changes one by one.

How should you start? You guess it right: you start with the implementor hierarchy. This time the `PriceType` interface needs some additional methods. The concrete implementors provide the implementation details. So, here is the `PriceType` interface with the new methods shown in bold:

```
// Implementor
interface PriceType {
    void displayProductPrice(String product,double cost);
    // Additional method(s) in implementation-2
    void festiveSeasonDiscount(int percentage);
    void sayThanks();
}
```

Since these additional methods are added in the `PriceType` interface, the concrete implementors need to implement these methods. Here is such an implementation. It is from the `OnlinePrice` class implementor:

```
class OnlinePrice implements PriceType{
    @Override
    public void displayProductPrice(String productType,
                                    double cost) {
        System.out.println("The " + productType + "'s
                           online cost is $" + cost*.9);
    }
    @Override
    public void festiveSeasonDiscount(int percentage) {
        System.out.println(" You can get a maximum of " +
                           percentage + "% discount in festive season.");
    }
    @Override
    public void sayThanks() {
        System.out.println("Thank you for your interest in our product
    }
}
```

Now concentrate on the abstraction hierarchy. Here `showPriceDetail()` can exist for both televisions and DVDs. So, you can place this method in the abstract class `ElectronicItem`. Note that since the method is NOT abstract now, there is no need to make this class abstract. But you still make the `ElectronicItem` class abstract, so that a client cannot instantiate this class directly. In addition, this time there are two new methods that call the corresponding implementor methods. Here is the `ElectronicItem` class for demonstration 2 with the important changes in bold:

```
// Abstraction
// Making the class abstract,
// so that you cannot instantiate it
// inside the client code.
abstract class ElectronicItem {
    protected String productType;
    protected double cost;
    // Composition - implementor
```

```

protected PriceType priceType;
protected ElectronicItem(PriceType priceType) {
    this.priceType = priceType;
}
protected void showPriceDetail() {
    priceType.displayProductPrice(productType, cost);
}
// Additional methods for demonstration-2
protected void giveDiscount() {
    priceType.festiveSeasonDiscount(5);
}
protected void conveyThanks() {
    priceType.sayThanks();
}
}

```

The subclasses of `ElectronicItem` can use all of these methods as-is, but they can provide additional functionalities as well. As per the new design, the `Television` class can use the `showPriceDetail()` method that is defined in its parent class. Also, it does not offer a double discount. So, you do not need to define any new method inside the following class:

```

class Television extends ElectronicItem{
    public Television(PriceType priceType) {
        super(priceType);
        this.productType="television";
        this.cost=2000;
    }
    // No additional methods exist for Television
}

```

A seller can provide double discounts on a DVD sold in a festive season and they can also send thanks message for the purchase. Note that the `DVD` class may need another method to provide the double discount (normal discount + additional discount). But you cannot add this method in the abstraction class `ElectronicItem`, because in that case, the `Television` class will also have this method. To accommodate this, in this implementation, add the following method in the `DVD` class:

```

// Specific method in the DVD class
public void getDoubleDiscountWithThanks() {
    getDiscount(10); //10% discount
    getDiscount(5); // Additional 5% discount
    conveyThanks();
}

```

NoteYou can see that inside the `getDoubleDiscountWithThanks()` method, you use the `getDiscount(...)` and `conveyThanks()` methods of `ElectronicItem`. This means you are coding in terms of “superclass abstraction.” This structure helps to vary the abstraction and implementation independently. In short, using this pattern, you maintain two separate but connected hierarchies that can vary independently.

Notice that a client he wants. This method is coded in the superclass abstraction and the other inheritance hierarchy is unaffected due to the addition of this DVD class. Let’s cross verify the key goal again.

Due to the addition of the DVD class in hierarchy-1, you do not need to change `ShowroomPrice` or `OnlinePrice`, which are placed in a different hierarchy (hierarchy-2). Similarly, if you add a separate class, say `SpecialDayPrice` in the `PriceType` interface in hierarchy-2, you do not need to make changes in hierarchy-1. Obviously, in this case, you ensure that `SpecialDayPrice` adheres to the `PriceType` contract.

NoteIn short, here you separate the methods that clients use from how these methods are implemented.

Now do a crosscheck of your understanding with the following class diagram. Then directly follow the complete demonstration and output. I don’t show the package explorer view for this modified implementation, which is easy to understand with the preceding discussions and the following class diagram.

Class Diagram

Figure [14-7](#) shows the modified class diagram. Again, this figure shows the most important parts of the class diagram for demonstration 2. I do not show the full method signatures in order to make the diagram short and simple. But an important note is added and separate hierarchies are marked so that you can notice the important parts. You can see that the `ElectronicItem`, `Television`, and `DVD` classes are placed in hierarchy-1 and the interface `PriceType` and the classes `OnlinePrice` and `ShowroomPrice` are placed in hierarchy-2.

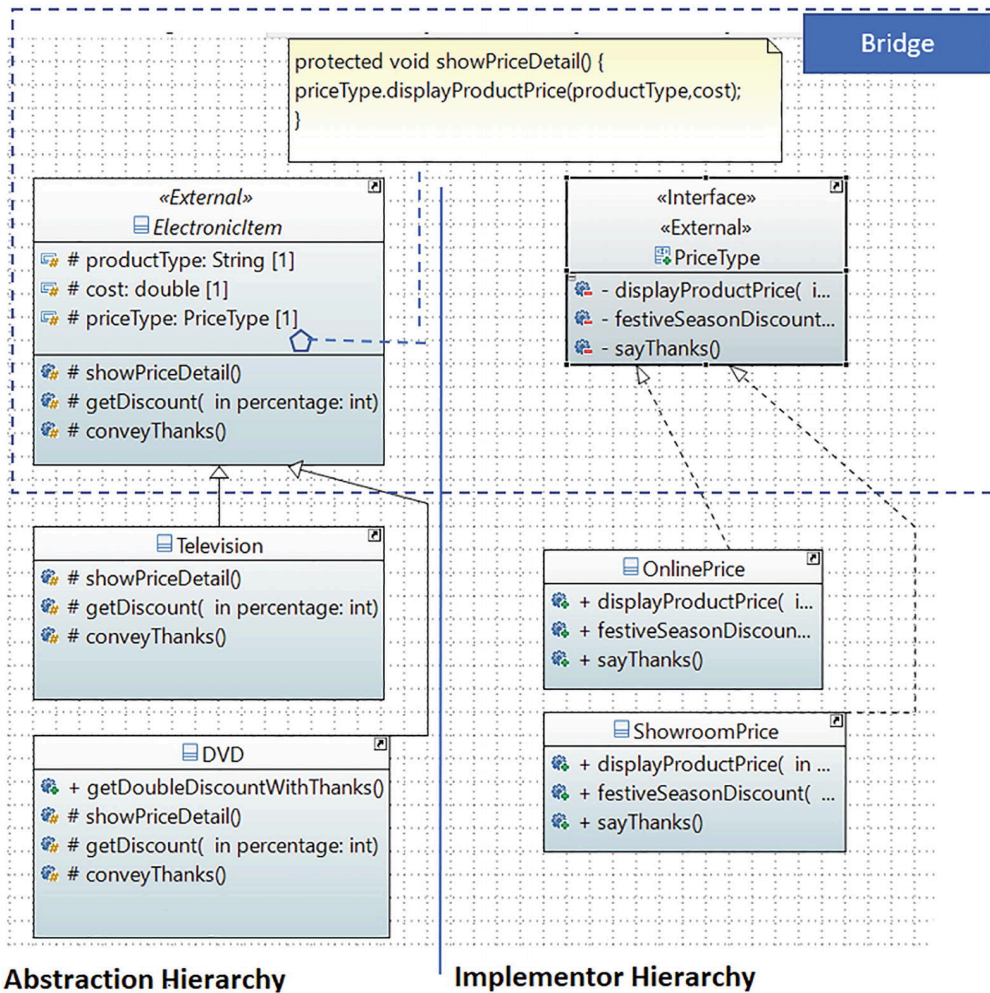


Figure 14-7 Class diagram for demonstration 2

Demonstration 2

Download the package `jdp3e.bridge.implementation_2` to see all parts of the modified program. Here is the complete implementation:

```

// Abstraction
// Making the class abstract,
// so that you cannot instantiate it
// ElectronicItem.java
abstract class ElectronicItem {
    protected String productType;
    protected double cost;
    // Composition - implementor
    protected PriceType priceType;
    protected ElectronicItem(PriceType priceType) {
        this.priceType = priceType;
    }
    protected void showPriceDetail() {
        priceType.displayProductPrice(productType, cost);
    }
    // The additional methods for demonstration-2
  
```



```

        protected void getDiscount(int percentage) {
            priceType.festiveSeasonDiscount(percentage);
        }
        protected void conveyThanks() {
            priceType.sayThanks();
        }
    }
    // Implementor
    // PriceType.java
    interface PriceType {
        void displayProductPrice(String product,double cost);
        // The additional method(s) in implementation-2
        void festiveSeasonDiscount(int percentage);
        void sayThanks();
    }
    // OnlinePrice.java
    class OnlinePrice implements PriceType{

        @Override
        public void displayProductPrice(String productType,
                                         double cost) {
            System.out.println("The " + productType + "'s
                               online cost is $" + cost*.9);
        }
        @Override
        public void festiveSeasonDiscount(int percentage) {
            System.out.println("You can get a maximum of " +
                               percentage + "% discount in festive season.");
        }
        @Override
        public void sayThanks() {
            System.out.println("Thank you for your interest in our product
        }
    }
    // ShowroomPrice.java
    class ShowroomPrice implements PriceType {
        @Override
        public void displayProductPrice(String productType,
                                         double cost) {
            System.out.println("The " + productType + "'s
                               showroom price is $" + cost);
        }
        @Override
        public void festiveSeasonDiscount(int percentage) {
            System.out.println("You can get a maximum of " +
                               percentage + "% discount in the

```

```

        festive seasons.");
    }
    @Override
    public void sayThanks() {
        System.out.println("Thank you for your interest in our product

    }
}
// Television.java
class Television extends ElectronicItem{
    /*
     * Implementation specific:
     * Delegating the task
     * to the Implementor object.
     */
    public Television(PriceType priceType) {
        super(priceType);
        this.productType="television";
        this.cost=2000;
    }
    // No additional method exists for Television
}
// DVD.java
class DVD extends ElectronicItem {
    /*
     * Implementation specific:
     * Delegating the task
     * to the Implementor object.
     */
    public DVD(PriceType priceType) {
        super(priceType);
        this.productType="DVD";
        this.cost=3000;
    }
    // Specific method in the DVD class

    public void getDoubleDiscountWithThanks() {
        getDiscount(10); //10% discount
        getDiscount(5); // Additional 5% discount
        conveyThanks();
    }
}
// Client.java
class Client {

```

```

public static void main(String[] args) {
    System.out.println("***Alternative Implementation
                        of Bridge Pattern.***");
    System.out.println("Verifying the market price of a television
    // Verifying online price
    ElectronicItem eItem = new Television(new
                                OnlinePrice());
    eItem.showPriceDetail();
    // Verifying the offline/showroom price
    eItem = new Television(new ShowroomPrice());
    eItem.showPriceDetail();
    eItem.getDiscount(7);
    eItem.conveyThanks();
    //Error: the following method is DVD specific.
    //eItem.getDoubleDiscountWithThanks();
    System.out.println("-----");
    System.out.println("Verifying the market price of a DVD.");
    // Verifying online price

    eItem = new DVD(new OnlinePrice());
    eItem.showPriceDetail();
    // Verifying the offline/showroom price
    eItem = new DVD(new ShowroomPrice());
    eItem.showPriceDetail();
    //Checking the DVD specific method
    ((DVD)eItem).getDoubleDiscountWithThanks();
}
}

```

Output

Here's the new output:

```

***Alternative Implementation of Bridge Pattern.***
Verifying the market price of a television.
The television's online cost is $1800.0
The television's showroom price is $2000.0
    You can get a maximum of 7% discount in the festive seasons.
Thank you for your interest in our product.
-----
Verifying the market price of a DVD.
The DVD's online cost is $2700.0
The DVD's showroom price is $3000.0
    You can get a maximum of 10% discount in the festive seasons.

```

You can get a maximum of 5% discount in the festive seasons.
Thank you for your interest in our product.

Q&A Session

14.1 How can this pattern make my programming life easier?

In this chapter, the two examples show you

- How to avoid the tight coupling between the items and their corresponding prices
- How to maintain two different hierarchies where both of them can be extended without impacting each other
- How to deal with multiple objects where implementations are shared among themselves

14.2 You could use simple subclassing instead of using this kind of design. Is this correct?

No. With simple subclassing, your implementations cannot vary dynamically. Your implementations may seem to behave differently, but actually they are bound to the abstraction at compile time.

14.3 Can I use getter-setter methods instead of a constructor inside the Abstraction class?

Yes. In fact, some developers prefer getter-setter methods over constructors, because you can set a new value for an attribute later. In the previous edition of this book, I showed both usages. In this book, I modified the example in demonstration 1 to focus on the required enhancement in demonstration 2.

14.4 What are the key advantages of using a Bridge design pattern?

Here are some advantages:

- Implementations are not bound to abstractions.
- The abstractions and implementations can grow independently.
- It simply means that the concrete classes are independent of the interface implementer classes. In other words, changes in one of them do not affect the other. So, you can also vary the abstraction and the implementation hierarchies in different ways.

14.5 What are the challenges associated with this pattern?

Here are some challenges:

- The overall structure is complex.
- Here you do not directly invoke a method. Instead, the abstraction layer delegates the work to the implementation layer. So, you may notice a slight performance impact when you execute an operation.

14.5 I see similarities between a Bridge design pattern and an Adapter design pattern. Am I right?

Sometimes the Bridge pattern is confused with the Adapter pattern—most accurately, with an Object Adapter design pattern. In cases like this, it is tough if you try to match a design pattern after seeing an implementation. Instead, it can be beneficial if you start from the beginning and try to understand the intent of the design.

Remember that the key purpose of an Adapter pattern is to deal with incompatible interfaces only and we assume that those interfaces already exist.

14.6 Do you mean that an adapter can be beneficial for me when I need to support legacy code but I want to implement the new features as well?

Yes.

14.7 When you explained the implementor interface before demonstration 1, you said, “...there need not be a one-to-one mapping between an abstraction class method and an implementor method.” Can you please elaborate?

See the following method definitions:

```
protected void getDiscount(int percentage) {  
    priceType.festiveSeasonDiscount(percentage);  
}  
protected void conveyThanks() {  
    priceType.sayThanks();  
}  
public void getDoubleDiscountWithThanks() {  
    getDiscount(10); // 10% discount
```

```
        getDiscount(5); // Additional 5% discount
        conveyThanks();
    }
```

From these method definitions, you can notice the following points:

- The `getDiscount()` method invokes the `festiveSeasonDiscount()` method and the `conveyThanks()` method invokes the `sayThanks()` method but these methods belong to a different inheritance hierarchy.
- Notice the `getDoubleDiscountWithThanks()` method now. Using this method, you invoke multiple methods from a different inheritance hierarchy.

14.8 “You can use a combination of the implementor method when you use an abstraction class method. In demonstration 2, you’ll see this clearly.” Can you please elaborate?

It is already shown. Notice the following method in demonstration 2:

```
// Specific method in the DVD class
public void getDoubleDiscountWithThanks() {
    getDiscount(10); //10% discount
    getDiscount(5); // Additional 5% discount
    conveyThanks();
}
```

You can see that when a client calls this method, it invokes two different implementor methods together.

Note Remember that a high-level abstraction method can include multiple implementor methods, but clients may not be aware of this.

Previous chapter

Next chapter

[< 13. Composite Pattern](#)

[15. Template Method Pattern >](#)